CHAPTER 13

Lifetimes and `unsafe` Code

Everywhere else in this book, the borrow checker checks your lifetimes for you. In `unsafe` code there are operations it does not check: dereferencing a raw pointer, calling an `unsafe fn` such as `slice::from_raw_parts`, implementing an `unsafe` trait. Lifetimes do not disappear when you use them. They move to the boundary. The function you build on top of `unsafe` code has a signature, safe callers are checked against that signature and nothing else, and so the signature must tell the truth about every borrow. *The signature is the contract* is never more literal than here: in safe code a wrong signature is rejected, while in `unsafe` code a wrong signature compiles and lets safe callers create dangling references.

This chapter shows where lifetimes enter and leave `unsafe` code. A raw pointer has no lifetime, and the reference you make from it gets whatever lifetime the surrounding code asks for, unless a signature bounds it. A struct that holds raw pointers needs `PhantomData` to tell the compiler which borrow it represents and how it varies. Some designs that look natural, such as a struct that borrows from itself, cannot be expressed at all. The drop check has an `unsafe` escape hatch that `Vec` uses and you cannot (yet). And one fact shapes several APIs: leaking a value is safe, so no safe API may depend on a destructor running.

Every `unsafe` block in this chapter's programs is sound, and says why in a `// SAFETY:` comment. Two programs contain an *unsound* function on purpose, to show that it compiles; they are marked as such and never call it.

In this chapter you will learn:

- why raw pointers carry no lifetime, how dereferencing one creates an *unbounded* lifetime, and how a signature bounds it;
- who promises the lifetime in APIs such as `slice::from_raw_parts`;
- what `PhantomData` is for: unused lifetime parameters (E0392), tying a raw pointer to a borrow, and choosing variance;
- why safe Rust rejects self-referential structs, what to do instead, and how `Pin` relates;
- how the drop check treats your types and `Vec`, and what the unstable `#[may_dangle]` promises;
- why leaking is safe, and why that is the reason `thread::scope` takes a closure;
- what Miri can and cannot tell you about your `unsafe` code.

13.1 Raw pointers carry no lifetime

A pointer is an address, not a borrow

A reference `&'a T` carries a region: the borrow checker knows where it is valid and checks every use against that. A raw pointer, `*const T` or `*mut T`, has no lifetime parameter at all. Creating one from a reference is safe, keeping it is safe, and the borrow checker stops tracking it the moment it is created: the pointer may outlive the value it points to, and nothing complains. Only dereferencing it requires `unsafe`, because only then does it matter whether the memory is still there. This compiles and runs, as part of `code/examples/ch13_raw_buf.rs`:

```
let pointer: *const String = {
    let temporary = String::from("temporary");
    &temporary // a `&String` coerced to a raw pointer: the borrow ends here
}; // `temporary` is dropped; `pointer` dangles, and that is allowed
assert!(!pointer.is_null()); // safe: this does not dereference it
```

That is exactly why raw pointers are useful for building data structures whose borrowing pattern the borrow checker cannot see, and exactly why they are dangerous: all the information that `'a` would have carried now lives only in your head, and in the `SAFETY:` comments you write down.

Unbounded lifetimes

Going back from a raw pointer to a reference, with `&*ptr` or with functions such as `slice::from_raw_parts`, produces a reference whose lifetime nothing constrains. The Rustonomicon calls it *unbounded* (https://doc.rust-lang.org/nomicon/unbounded-lifetimes.html): "Such a lifetime becomes as big as context demands." If the context wants `'static`, it gets `'static`.

Here is a buffer of bytes held as a raw pointer and a length, the shape you get from a C library or a custom allocator. (With only the standard library, the memory comes from a `Box<[u8]>`, so that the `Drop` impl can free it again.) The rest of `code/examples/ch13_raw_buf.rs` defines it:

```
/// An owned byte buffer kept as a raw pointer and a length, as you might get
/// from a C library or a custom allocator.
pub struct RawBuf {
    ptr: *mut u8,
    len: usize,
}

impl RawBuf {
    pub fn new(bytes: &[u8]) -> RawBuf {
        let boxed: Box<[u8]> = bytes.into();
        let len = boxed.len();
        // `into_raw` gives up ownership; `Drop` below takes it back.
        let ptr = Box::into_raw(boxed).cast::<u8>();
        RawBuf { ptr, len }
    }
}

impl Drop for RawBuf {
    fn drop(&mut self) {
        let slice = std::ptr::slice_from_raw_parts_mut(self.ptr, self.len);
        // SAFETY: `ptr` and `len` came from `Box::into_raw` in `new`, and this
        // is the only place that turns them back into a box, exactly once.
        drop(unsafe { Box::from_raw(slice) });
    }
}
```

Now two ways to look at the bytes. The first has an elided signature, which ties the slice to the borrow of `self` (Chapter 3):

```
impl RawBuf {
    /// The elided signature reads `fn as_bytes<'a>(&'a self) -> &'a [u8]`.
    pub fn as_bytes(&self) -> &[u8] {
        // SAFETY: `ptr` points to `len` initialized bytes owned by `self`.
        // They stay allocated while `self` is alive, and no method mutates
        // them, so they are valid and unchanged for the lifetime of `&self`,
        // which is the lifetime the signature gives the slice.
        unsafe { slice::from_raw_parts(self.ptr, self.len) }
    }
}
```

The second names a lifetime that appears only in the output, so the *caller* picks it, as for any lifetime parameter (Chapter 9, *Who picks the lifetime?*):

```
impl RawBuf {
    /// UNSOUND: the caller picks `'a`, and nothing ties it to `self`.
    #[allow(dead_code)] // only used by `keep_forever`, which is never called
    pub fn as_bytes_unbounded<'a>(&self) -> &'a [u8] {
        // SAFETY: (incomplete) the bytes are valid while `self` is alive, but
        // this signature lets the caller keep the slice longer than that.
        unsafe { slice::from_raw_parts(self.ptr, self.len) }
    }
}

/// Compiles, and returns a dangling slice: `buf` is freed at the end of the
/// function. Never called: calling it is undefined behavior.
#[allow(dead_code)]
fn keep_forever() -> &'static [u8] {
    let buf = RawBuf::new(b"temporary");
    buf.as_bytes_unbounded()
}
```

The bodies are identical, and both compile, because `from_raw_parts` will return a slice of any lifetime you like. The difference is entirely in the contract. With `as_bytes_unbounded`, the function `keep_forever` compiles: it asks for `&'static [u8]`, the signature grants it, and the buffer is freed when `buf` goes out of scope at the end of the function. Any use of the returned slice reads freed memory. That is undefined behavior, so the program never calls `keep_forever`; it exists to show that the compiler accepts it.

With the bounded `as_bytes`, the same function is rejected (`code/compile_fail/ch13_bounded_slice.rs`):

```
pub fn keep_forever() -> &'static [u8] {
    let buf = RawBuf::new(b"temporary");
    buf.as_bytes()
}
```

```
error[E0515]: cannot return value referencing local variable `buf`
  --> compile_fail/ch13_bounded_slice.rs:37:5
   |
37 |     buf.as_bytes()
   |     ---^^^^^^^^^^
   |     |
   |     returns a value referencing data owned by the current function
   |     `buf` is borrowed here
```

E0515 is the ordinary "returning a reference to a local" error of Chapter 5, "returns a value referencing data owned by the current function". The `unsafe` block inside `as_bytes` did not weaken anything:

once the signature says `&'a self -> &'a [u8]`, the borrow checker enforces it on every caller. The correct program uses the slice only while the buffer lives:

```
let buf = RawBuf::new(b"listen port 8080");
let bytes = buf.as_bytes(); // borrows `buf`
let first_word = bytes.split(|&b| b == b' ').next().unwrap_or(&[]);
println!("{}", String::from_utf8_lossy(first_word));
drop(buf); // fine: `bytes` and `first_word` are no longer used
```

```
listen
```

The Rustonomicon's advice is the lesson of this section: "The easiest way to avoid unbounded lifetimes is to use lifetime elision at the function boundary." Bound the lifetime as soon as possible, ideally in the same function that dereferences the pointer, by giving that function a signature that ties the result to the owner of the memory.

Who promises the lifetime?

Several standard library functions return references with unbounded lifetimes. Here are four, from the 1.99 source, without their `const` qualifiers:

Function	Signature	Lifetime of the result
`slice::from_raw_parts`	`unsafe fn from_raw_parts<'a, T>(data: *const T, len: usize) -> &'a [T]`	picked by the caller
`<*const T>::as_ref`	`unsafe fn as_ref<'a>(self) -> Option<&'a T>`	picked by the caller
`NonNull::as_ref`	`unsafe fn as_ref<'a>(&self) -> &'a T`	picked by the caller, not tied to `&self`
`str::from_utf8_unchecked`	`unsafe fn from_utf8_unchecked(v: &[u8]) -> &str`	elided: tied to `v`

In the first three, `'a` appears only in the result. The compiler cannot check it, so the documentation turns it into a precondition that the caller promises. For `from_raw_parts`, one of the safety requirements reads: "The memory referenced by the returned slice must not be mutated for the duration of lifetime `'a`, except inside an `UnsafeCell`." And a *Caveat* section gives the same advice as the Rustonomicon:

> The lifetime for the returned slice is inferred from its usage. To prevent accidental misuse, it's suggested to tie the lifetime to whichever source lifetime is safe in the context, such as by providing a helper function taking the lifetime of a host value for the slice, or by explicit annotation.

`RawBuf::as_bytes` is such a helper: its host value is `self`. The fourth row shows the other style. `from_utf8_unchecked` takes a reference, not a pointer, so elision bounds its result to `v`, and the only promise left to the caller is that the bytes are UTF-8.

Note the third row: `NonNull::as_ref` takes `&self` and still returns an unbounded lifetime. The `&self` is a borrow of the *pointer*, which says nothing about how long the memory it points to stays valid; tying the result to it would be wrong in both directions. Read signatures of `unsafe` functions with this in mind: an elided-looking `&self` is not always where the lifetime comes from.

13.2 `PhantomData`

Unused lifetime parameters: E0392

Suppose we want our own slice iterator, built from raw pointers the way the standard library's `slice::Iter` is. It hands out `&'a T`, so the type needs a lifetime parameter `'a`; but its fields are a raw pointer and a count, which carry no lifetime (`code/compile_fail/ch13_unused_lifetime.rs`):

```
pub struct RawIter<'a, T> {
    ptr: *const T,
    remaining: usize,
}
```

```
error[E0392]: lifetime parameter `'a` is never used
 --> compile_fail/ch13_unused_lifetime.rs:6:20
  |
6 | pub struct RawIter<'a, T> {
  |                    ^^ unused lifetime parameter
  |
  = help: consider removing `'a`, referring to it in a field, or using a marker such as
`PhantomData`
```

E0392 says "lifetime parameter `'a` is never used". The compiler insists for a good reason: the meaning of a lifetime parameter, and the *variance* of the type in it (Chapter 4), are computed from the fields. With no field mentioning `'a`, there is nothing to compute from: `RawIter<'a, T>` could be shrunk, grown, or neither, and the compiler would have to guess. The help lists the options, and the third one is the tool for this job: "a marker such as `PhantomData`".

Tying a raw pointer to a borrow

`PhantomData<X>` is a zero-sized type that holds no data. It tells the compiler to treat the containing type *as if* it held a value of type `X`: for variance, for auto traits such as `Send`, for the drop check, and for implied bounds. Here is the iterator with a marker that says "this type borrows values of type `T` for `'a`" (`code/examples/ch13_raw_iter.rs`):

```
/// Iterates over a slice with a raw pointer, like `slice::Iter` does.
pub struct RawIter<'a, T> {
    ptr: *const T,
    remaining: usize,
    /// "This type borrows `T`s for `'a`": no data, only a lifetime and a
    /// variance for the compiler.
    _borrow: PhantomData<&'a T>,
}

impl<'a, T> RawIter<'a, T> {
    pub fn new(slice: &'a [T]) -> Self {
        RawIter {
            ptr: slice.as_ptr(),
            remaining: slice.len(),
            _borrow: PhantomData,
        }
    }
}

impl<'a, T> Iterator for RawIter<'a, T> {
    type Item = &'a T;

    fn next(&mut self) -> Option<&'a T> {
        if self.remaining == 0 {
            return None;
```

```
        }
        // SAFETY: `ptr` points into the slice passed to `new`, at an element
        // not yet returned (`remaining > 0`). The slice is borrowed for `'a`,
        // so it is neither freed nor mutated while `'a` lasts, and returning
        // `&'a T` is exactly what that borrow allows.
        let item = unsafe { &*self.ptr };
        // SAFETY: there are `remaining` elements from `ptr` on, so `ptr + 1`
        // stays within the slice or one past its end.
        self.ptr = unsafe { self.ptr.add(1) };
        self.remaining -= 1;
        Some(item)
    }
}
```

```
let words = vec![String::from("listen"), String::from("port")];
let longest = RawIter::new(&words).max_by_key(|w| w.len());
println!("{longest:?}");
```

```
Some("listen")
```

The `SAFETY:` comment of `next` relies on one fact: the slice stays borrowed, and therefore alive and unmodified, for `'a`. The signature of `new` is what makes that fact true. It takes a `&'a [T]` and returns a `RawIter<'a, T>`, so the borrow checker keeps the slice borrowed for as long as the iterator, or any item it returned, is in use (`code/compile_fail/ch13_raw_iter_borrow.rs`):

```
pub fn caller() {
    let mut words = vec![String::from("listen"), String::from("port")];
    let first = RawIter::new(&words).next();
    words.push(String::from("8080")); // may reallocate: `first` would dangle
    println!("{first:?}");
}
```

```
error[E0502]: cannot borrow `words` as mutable because it is also borrowed as immutable
  --> compile_fail/ch13_raw_iter_borrow.rs:43:5
   |
42 |     let first = RawIter::new(&words).next();
   |                              ------ immutable borrow occurs here
43 |     words.push(String::from("8080")); // may reallocate: `first` would dangle
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ mutable borrow occurs here
44 |     println!("{first:?}");
   |                -----
   |                |
   |                immutable borrow later used here
```

This is the E0502 of Chapter 1, and that is the point: a type built on raw pointers gets the same protection as `slice::Iter`, because its signature and its marker say the same thing a `&'a T` field would. The standard library's `slice::Iter<'a, T>` has exactly this field, `_marker: PhantomData<&'a T>`, next to its raw pointers.

Choosing variance

The marker also decides the variance of the type, which is the part that is easy to get wrong. Here are the common choices, with what they mean for a lifetime `'a` and a type `T` (the auto-trait column was checked with rustc 1.99):

Marker	Variance	Also
`PhantomData<&'a T>`	covariant in `'a` and `T`	requires `T: 'a`; `Send` and `Sync` like `&T`
`PhantomData<&'a mut T>`	covariant in `'a`, invariant in `T`	requires `T: 'a`
`PhantomData<T>`	covariant in `T`	"owns a `T`" for the drop check
`PhantomData<*const T>`	covariant in `T`	not `Send`, not `Sync`
`PhantomData<*mut T>`	invariant in `T`	not `Send`, not `Sync`
`PhantomData<fn(T)>`	contravariant in `T`	always `Send` and `Sync`
`PhantomData<fn() -> T>`	covariant in `T`	always `Send` and `Sync`; owns nothing
`PhantomData<fn(&'a ()) -> &'a ()>`	invariant in `'a`	always `Send` and `Sync`
`PhantomData<Cell<&'a ()>>`	invariant in `'a`	not `Sync`

To see the difference, here are three handle types that differ only in their marker (`code/examples/ch13_variance_markers.rs`):

```
/// Covariant in `'a`, like `&'a T`.
pub struct Shared<'a> {
    _marker: PhantomData<&'a ()>,
}

/// Contravariant in `'a`, like the argument of `fn(&'a ())`.
pub struct Sink<'a> {
    _marker: PhantomData<fn(&'a ())>,
}

/// Invariant in `'a`: the argument and the result of a function at once.
pub struct Exact<'a> {
    _marker: PhantomData<fn(&'a ()) -> &'a ()>,
}
```

```
/// Covariance: a handle for a longer region can be used where a shorter one
/// is expected.
fn shrink<'long: 'short, 'short>(handle: Shared<'long>) -> Shared<'short> {
    handle
}

/// Contravariance: the other way around.
fn grow<'long: 'short, 'short>(sink: Sink<'short>) -> Sink<'long> {
    sink
}

/// Invariance: only the same region.
fn same<'a>(exact: Exact<'a>) -> Exact<'a> {
    exact
}
```

```
covariant, contravariant and invariant handles all compile
```

All three conversions compile. Trying to shrink the invariant one does not (`code/compile_fail/ch13_invariant_marker.rs`):

```
pub fn shrink<'long: 'short, 'short>(exact: Exact<'long>) -> Exact<'short> {
    exact
}
```

```
error: lifetime may not live long enough
  --> compile_fail/ch13_invariant_marker.rs:13:5
   |
12 | pub fn shrink<'long: 'short, 'short>(exact: Exact<'long>) -> Exact<'short> {
   |               -----          ------ lifetime `'short` defined here
   |               |
   |               lifetime `'long` defined here
13 |     exact
   |     ^^^^^ function was supposed to return data with lifetime `'long` but it is
returning data with lifetime `'short`
   |
   = help: consider adding the following bound: `'short: 'long`
   = note: requirement occurs because of the type `Exact<'_>`, which makes the generic
argument `'_` invariant
   = note: the struct `Exact<'a>` is invariant over the parameter `'a`
   = help: see <https://doc.rust-lang.org/nomicon/subtyping.html> for more information about
variance
```

The main message, "function was supposed to return data with lifetime `'long` but it is returning data with lifetime `'short`", sounds backwards until you remember what invariance means: `Exact<'long>` and `Exact<'short>` are unrelated types, so returning one where the other is expected would need the two regions to be *equal*, and the help offers the missing half, `'short: 'long`. The notes then name the cause: "the struct `Exact<'a>` is invariant over the parameter `'a`". When you see this note about a type of your own, look at its `PhantomData`.

When variance matters: a write handle on `NonNull`

Which marker to choose follows the rule of Chapter 4: a type through which you can *write* a `T` must be invariant in `T`, or a caller could shrink the type, write something short-lived, and leave a long-lived owner holding a dangling reference. `Cell<&'a str>` is invariant for this reason. A raw-pointer type gets no such protection automatically, and the documentation of `NonNull` warns about it:

> Unlike `*mut T`, `NonNull<T>` is covariant over `T`. This is usually the correct choice for most data structures and safe abstractions, such as `Box`, `Rc`, `Arc`, `Vec`, and `LinkedList`.
>
> In rare cases, if your type exposes a way to mutate the value of `T` through a `NonNull<T>`, and you need to prevent unsoundness from variance (for example, if `T` could be a reference with a shorter lifetime), you should add a field to make your type invariant, such as `PhantomData<Cell<T>>` or `PhantomData<&'a mut T>`.

Here is such a type, written first with the wrong marker (`code/examples/ch13_covariant_slot.rs`). `CovariantSlot<'s, 'v>` is a handle that overwrites a `&'v str` owned by someone else:

```rust
/// UNSOUND: covariant in `'v` (through `NonNull`), yet it writes a `&'v str`.
pub struct CovariantSlot<'s, 'v> {
    target: NonNull<&'v str>,
    _borrow: PhantomData<&'s ()>, // mentions `'s` only
}
```

Its marker mentions only `'s`, so its variance in `'v` comes from `NonNull<&'v str>`: covariant. This function compiles:

```rust
/// Compiles, and returns a dangling `&'static str`. Never called: calling it
/// is undefined behavior.
#[allow(dead_code)]
```

```
fn dangling() -> &'static str {
    let mut name: &'static str = "static";
    {
        let mut slot = CovariantSlot::new(&mut name);
        let temporary = String::from("temporary");
        slot.set(&temporary); // accepted: `'v` shrinks to fit `temporary`
    }
    name // points to the freed `temporary`
}
```

`name` is a `&'static str`, so the slot starts as a `CovariantSlot<'_, 'static>`. Covariance lets the compiler shrink `'v` to the lifetime of `temporary`, `set` accepts `&temporary`, and `name` ends up pointing to a freed string, while its type still says `'static`. As before, the program never calls this function.

The correct type uses the marker the documentation recommends (`code/examples/ch13_out_slot.rs`):

```
/// A handle that can overwrite a `&'v str` owned by someone else, for `'s`.
/// (In safe code you would store the `&'s mut &'v str` itself; the raw pointer
/// stands for one that came from C, or from a node of a linked structure.)
pub struct OutSlot<'s, 'v> {
    target: NonNull<&'v str>,
    /// Covariant in `'s`, invariant in `'v`, like `&'s mut &'v str`.
    _borrow: PhantomData<&'s mut &'v str>,
}

impl<'s, 'v> OutSlot<'s, 'v> {
    pub fn new(target: &'s mut &'v str) -> Self {
        OutSlot {
            target: NonNull::from(target),
            _borrow: PhantomData,
        }
    }

    pub fn set(&mut self, value: &'v str) {
        // SAFETY: `target` came from a `&'s mut &'v str` that this handle
        // keeps borrowed for `'s`, so the slot is valid and nobody else can
        // access it. `value` lives for `'v`, the lifetime the slot's owner
        // expects; invariance in `'v` stops anyone from shrinking `'v` first.
        unsafe { *self.target.as_ptr() = value }
    }
}
```

```
let mut setting: &str = "unset";
{
    let mut slot = OutSlot::new(&mut setting);
    slot.set("listen");
}
println!("{setting}");
```

```
listen
```

`PhantomData<&'s mut &'v str>` makes `OutSlot` behave like the `&'s mut &'v str` it stands for: covariant in `'s`, invariant in `'v`. The dangerous function is now rejected (`code/compile_fail/ch13_out_slot_variance.rs`):

```
pub fn dangling() -> &'static str {
    let mut name: &'static str = "static";
    {
        let mut slot = OutSlot::new(&mut name);
        let temporary = String::from("temporary");
        slot.set(&temporary);
    }
```

```
    name
}
```

```
error[E0597]: `temporary` does not live long enough
  --> compile_fail/ch13_out_slot_variance.rs:33:18
   |
29 |     let mut name: &'static str = "static";
   |                   ------------ type annotation requires that `temporary` is borrowed
for `'static`
...
32 |         let temporary = String::from("temporary");
   |             --------- binding `temporary` declared here
33 |         slot.set(&temporary);
   |                  ^^^^^^^^^^ borrowed value does not live long enough
34 |     }
   |     - `temporary` dropped here while still borrowed
```

The label "type annotation requires that `temporary` is borrowed for `'static`" points at `name`'s type. With invariance, `'v` must stay exactly `'static`, so `set` demands a `&'static str`, and `&temporary` cannot be one. The `SAFETY:` comment of `set` says that invariance is part of the argument; with the wrong marker, the same comment would have been a false statement that no compiler could catch.

13.3 Self-referential structs

Why safe Rust rejects them

A document that keeps its text and a reference to its title line *inside that text* is a natural thing to want. Here is the attempt everyone writes once (`code/compile_fail/ch13_self_ref_new.rs`; it produces two errors):

```
pub struct Document<'a> {
    text: String,
    title: &'a str, // meant to point into `text`
}

impl<'a> Document<'a> {
    pub fn new(text: String) -> Document<'a> {
        let title = text.lines().next().unwrap_or("");
        Document { text, title }
    }
}
```

```
error[E0515]: cannot return value referencing function parameter `text`
  --> compile_fail/ch13_self_ref_new.rs:14:9
   |
13 |         let title = text.lines().next().unwrap_or("");
   |                     ---- `text` is borrowed here
14 |         Document { text, title }
   |         ^^^^^^^^^^^^^^^^^^^^^^^^ returns a value referencing data owned by the current
function

error[E0505]: cannot move out of `text` because it is borrowed
  --> compile_fail/ch13_self_ref_new.rs:14:20
   |
11 | impl<'a> Document<'a> {
   |      -- lifetime `'a` defined here
12 |     pub fn new(text: String) -> Document<'a> {
   |                ---- binding `text` declared here
13 |         let title = text.lines().next().unwrap_or("");
   |                     ---- borrow of `text` occurs here
```

```
14 |         Document { text, title }
   |         -----------^^^^---------
   |         |          |
   |         |          move out of `text` occurs here
   |         returning this value requires that `text` is borrowed for `'a`
```

E0515 says that `new` returns a value referencing its parameter `text`, which the function owns. E0505 adds that `text` is moved into the struct while it is still borrowed by `title`. Both are the compiler's way of saying that `'a` has no correct value. It would have to be "the lifetime of the `Document` that contains this very field", and no such lifetime exists: lifetime parameters are picked by whoever *uses* the type, before the value exists. (Writing `&'self str` does not help either; `'self` is not a lifetime name Rust knows.)

Curiously, the same struct can be built in place (`code/compile_fail/ch13_self_ref_move.rs`):

```rust
pub fn load() {
    let mut doc = Document {
        text: String::from("Config\nport 8080"),
        title: "",
    };
    doc.title = doc.text.lines().next().unwrap_or("");
    println!("{}", doc.title); // fine: used in place
    let moved = doc; // a move would leave `title` pointing at the old place
    println!("{}", moved.title);
}
```

```
error[E0505]: cannot move out of `doc` because it is borrowed
  --> compile_fail/ch13_self_ref_move.rs:18:17
   |
12 |     let mut doc = Document {
   |         ------- binding `doc` declared here
...
16 |     doc.title = doc.text.lines().next().unwrap_or("");
   |                 -------- borrow of `doc.text` occurs here
17 |     println!("{}", doc.title); // fine: used in place
18 |     let moved = doc; // a move would leave `title` pointing at the old place
   |                 ^^^
   |                 |
   |                 move out of `doc` occurs here
   |                 borrow later used here
```

The assignment to `doc.title` and the first `println!` are accepted: `doc` is borrowed by one of its own fields, and the borrow checker treats it like any other borrow. The price is that `doc` is frozen. The move on the next line is E0505, with an unusual pair of labels on the same spot: "move out of `doc` occurs here" and "borrow later used here", because moving `doc` also moves the borrow stored in `doc.title`. A mutation such as `doc.text.push_str(...)` followed by a use of `doc.title` is rejected too (as E0502).

Both restrictions protect real invariants. A move copies the bytes of the struct to a new place, so a reference into the old place would dangle whenever the referenced data lives *inside* the struct (an array field, say). For a `String`, the characters live on the heap and do not move, but a `push_str` may reallocate them, which would leave `title` dangling just the same. The borrow checker does not know which case it is looking at, so it forbids both.

Indices instead of references

The standard alternative stores *positions* instead of references, and borrows on demand (`code/examples/ch13_document_ranges.rs`):

```
pub struct Document {
    text: String,
    title: Range<usize>, // byte positions in `text`, not a reference
}

impl Document {
    pub fn new(text: String) -> Document {
        let end = text.find('\n').unwrap_or(text.len());
        Document {
            text,
            title: 0..end,
        }
    }

    /// Elided: `fn title<'a>(&'a self) -> &'a str`, a borrow of the document.
    pub fn title(&self) -> &str {
        &self.text[self.title.clone()]
    }

    /// Mutation is allowed, and must keep the positions valid.
    pub fn append(&mut self, more: &str) {
        self.text.push_str(more); // may reallocate: a `&str` field would dangle
    }
}
```

```
let mut doc = Document::new(String::from("Config\nport 8080"));
println!("{}", doc.title());
doc.append("\nlog-level debug");
let moved = doc; // moving is fine: a range does not point anywhere
println!("{} ({} bytes)", moved.title(), moved.text.len());
```

```
Config
Config (32 bytes)
```

A `Range<usize>` does not point anywhere, so moving the document is free, and `append` may reallocate the text without invalidating anything. The borrow appears only in `title(&self) -> &str`, where elision ties it to the borrow of the document: exactly the bounded lifetime of the previous section, without any `unsafe`. The cost is a bounds check when slicing, and the duty to keep the positions consistent with the text (here, `append` only adds at the end). The same idea scales up: arenas with index handles, graph nodes stored in a `Vec` and referred to by number, string interners that hand out ids. Chapter 14 returns to the alternative of keeping the owner and the borrowing view in two separate types.

How `Pin` relates

Chapter 11 showed that the compiler *does* build self-referential values: an async block that keeps a reference to its own local across an `.await`. They are sound because they are polled through `Pin<&mut Self>`, which promises that the value will not move again. `Pin` is that promise and nothing more: it does not give safe code a way to create a self-reference. Building a self-referential type by hand still means storing a raw pointer into the value, setting it after the value has reached its final place, preventing moves with `Pin`, and writing the `SAFETY:` arguments for every access. That is possible,

and it is a job for a small, carefully reviewed library type, not for application code. Before you start, check whether indices, or splitting the owner from the view, would do.

13.4 The drop check and `#[may_dangle]`

Chapter 5 introduced the drop check: when a value is dropped, any borrow its destructor might use must still be valid. Here is the rule seen from the side of a type author (`code/compile_fail/ch13_dropck_log.rs`):

```
pub struct Log<T> {
    entries: Vec<T>,
}

impl<T> Drop for Log<T> {
    fn drop(&mut self) {
        println!("dropping a log of {} entries", self.entries.len());
    }
}

pub fn record() {
    let mut log = Log {
        entries: Vec::new(),
    };
    let line = String::from("GET /index.html");
    log.entries.push(&line);
}
```

```
error[E0597]: `line` does not live long enough
  --> compile_fail/ch13_dropck_log.rs:21:22
   |
20 |     let line = String::from("GET /index.html");
   |         ---- binding `line` declared here
21 |     log.entries.push(&line);
   |                      ^^^^^ borrowed value does not live long enough
22 | }
   | -
   | |
   | `line` dropped here while still borrowed
   | borrow might be used here, when `log` is dropped and runs the `Drop` code for type
`Log`
   |
   = note: values in a scope are dropped in the opposite order they are defined
```

Locals are dropped in the reverse order of declaration, so `line` is dropped before `log`. When `log` is dropped, its `Drop` impl runs, and for all the compiler knows, `drop` could read the `&String` entries, after `line` is gone. The label says precisely that: "borrow might be used here, when `log` is dropped and runs the `Drop` code for type `Log`". Our `drop` only reads the length, but the drop check does not look into the body of `drop`. A generic `Drop` impl is a signature like any other, and it says "may use `T`".

Yet a plain `Vec<&String>` in the same position is fine (`code/examples/ch13_dropck.rs`):

```
fn record_in_vec() -> usize {
    let mut entries = Vec::new();
    let line = String::from("GET /index.html");
    entries.push(&line);
    entries.len()
} // `line` is dropped first, then `entries`: accepted
```

`Vec` has a `Drop` impl too. The difference is a promise written into it. In the standard library's source, it reads like this (simplified: the 1.99 source also carries `const` annotations):

```
unsafe impl<#[may_dangle] T, A: Allocator> Drop for Vec<T, A> {
    fn drop(&mut self) { /* drops the elements, then the buffer is freed */ }
}
```

`#[may_dangle] T` tells the drop check: "when this `drop` runs, values of type `T` may already be dangling; I will not use them, except to drop them". The impl must be `unsafe impl`, because the compiler takes the promise on trust: a `drop` that printed its elements would break it. Together with the promise, the standard library keeps a `PhantomData<T>` in the buffer type behind `Vec`, which tells the drop check that dropping a `Vec<T>` drops `T` values. That matters when `T` itself has a destructor that uses borrows: then the strict rule applies again, through `T`. For a `Vec<&String>`, dropping a `&String` does nothing, so the borrow may dangle.

The attribute is unstable. On stable Rust 1.99 it is E0658 (`code/compile_fail/ch13_may_dangle_unstable.rs`):

```
// SAFETY: `drop` never reads or writes a `T`; it only reads the length.
unsafe impl<#[may_dangle] T> Drop for Log<T> {
    fn drop(&mut self) {
        println!("dropping a log of {} entries", self.entries.len());
    }
}
```

```
error[E0658]: the `may_dangle` attribute has unstable semantics and may be removed in the
future
  --> compile_fail/ch13_may_dangle_unstable.rs:10:15
   |
10 | unsafe impl<#[may_dangle] T> Drop for Log<T> {
   |               ^^^^^^^^^^
   |
   = note: see issue #34761 <https://github.com/rust-lang/rust/issues/34761> for more
information
```

With `#![feature(dropck_eyepatch)]`, the nightly compiler of 2026-10-08 accepts this impl, and with it the `record` function of `ch13_dropck_log.rs`. On stable, the usual workaround is to keep the `Drop` impl away from `T`: put the cleanup in a field whose type does not mention `T`, and give the generic type no `Drop` impl of its own:

```
/// The `Drop` lives in a field that holds no `T`.
struct Report {
    count: usize,
}

impl Drop for Report {
    fn drop(&mut self) {
        println!("dropping a log of {} entries", self.count);
    }
}

pub struct Log<T> {
    entries: Vec<T>,
    report: Report,
}

impl<T> Log<T> {
    pub fn new() -> Self {
```

```
        Log {
            entries: Vec::new(),
            report: Report { count: 0 },
        }
    }

    pub fn push(&mut self, entry: T) {
        self.entries.push(entry);
        self.report.count = self.entries.len();
    }
}

fn record_in_log() {
    let mut log = Log::new();
    let line = String::from("GET /index.html");
    log.push(&line);
} // `Log<&String>` has no `Drop` impl of its own: accepted
```

```
vec: 1 entries
dropping a log of 1 entries
```

Dropping a `Log<&String>` now drops its fields: a `Vec<&String>`, which is lenient thanks to `#[may_dangle]`, and a `Report`, which holds no borrow. Nothing in the drop glue can use the dangling reference, and the compiler can see it. As a user of collections, the rule to remember is short: standard containers (`Vec`, `Box`, `HashMap`, ...) may hold borrows that die just before them; your own generic type with a `Drop` impl may not, unless the `Drop` is moved out of its way.

13.5 Leaking is safe

`mem::forget`, `Box::leak` and cycles

Safe Rust lets you skip a destructor in several ways (`code/examples/ch13_leaks.rs`):

```
struct Connection {
    name: &'static str,
}

impl Drop for Connection {
    fn drop(&mut self) {
        println!("closing {}", self.name);
    }
}
```

```
/// A node that may point to another node.
struct Node {
    _connection: Connection,
    next: RefCell<Option<Rc<Node>>>,
}
```

```
drop(Connection { name: "dropped" }); // runs `drop`
mem::forget(Connection { name: "forgotten" }); // never runs it

let leaked: &'static mut Connection =
    Box::leak(Box::new(Connection { name: "leaked" }));
println!("{} lives until the end of the program", leaked.name);

let first = Rc::new(Node {
    _connection: Connection { name: "in a cycle" },
    next: RefCell::new(None),
});
```

```
*first.next.borrow_mut() = Some(Rc::clone(&first)); // points to itself
drop(first); // the count drops from 2 to 1: never 0, never freed
println!("end of main");
```

```
closing dropped
leaked lives until the end of the program
end of main
```

Only the first connection is closed. `mem::forget` takes ownership and never runs `drop`. `Box::leak` turns a box into a reference that may have any lifetime, `'static` included, because the memory is simply never freed (Chapter 7 uses it as a source of `&'static`); its documentation says "the type `T` must outlive the chosen lifetime `'a`", which is the only requirement left. And a reference cycle of `Rc` handles keeps its own count above zero after the last outside handle is dropped, so its destructor never runs either, with no special function involved at all.

None of this needs `unsafe`, and the documentation of `mem::forget` explains why it does not:

> `forget` is not marked as `unsafe`, because Rust's safety guarantees do not include a guarantee that destructors will always run.

and draws the consequence for everyone who writes `unsafe` code:

> Because forgetting a value is allowed, any `unsafe` code you write must allow for this possibility. You cannot return a value and expect that the caller will necessarily run the value's destructor.

Why `thread::scope` takes a closure

This rule was learned the hard way. In the months before Rust 1.0, the standard library had a function `thread::scoped` that returned a `JoinGuard`, described in the 1.0 documentation as "An RAII-style guard that will block until thread termination when dropped" (https://doc.rust-lang.org/1.0.0/std/thread/index.html). The scoped thread could borrow the caller's locals, and the guard's destructor joined it before those locals could die. RFC 1066, "safe `mem::forget`" (https://rust-lang.github.io/rfcs/1066-safe-mem-forget.html), records what went wrong: "It was recently discovered by @arielb1 that the `thread::scoped` API was unsound", because "the safety of `thread::scoped` relied on the fact that the `Drop` implementation for `JoinGuard` was *always* run", and a cycle of `Rc` handles could leak the guard without any `unsafe` code. Here is the idea as a sketch (not the historical code, and not compiled):

```
let data = vec![1, 2, 3];
let guard = thread::scoped(|| println!("{data:?}")); // the thread borrows `data`
mem::forget(guard); // the destructor that would join never runs
drop(data);         // the thread may still be reading `data`
```

The RFC made `mem::forget` safe and stated that not running destructors is not unsafe. `thread::scoped` shipped in Rust 1.0 only as an unstable function behind a feature gate, and today's standard library no longer has it. Scoped threads came back, stable, in Rust 1.63, with a design that does not depend on any destructor:

```
pub fn scope<'env, F, T>(f: F) -> T
where
    F: for<'scope> FnOnce(&'scope Scope<'scope, 'env>) -> T,
```

The join is not in a guard that the caller holds. It is in the body of `scope`, which calls `f`, then waits for every thread spawned through the `Scope`, then returns. There is no value the caller could forget, because the caller does not get control back until the threads are done (`code/examples/ch13_scope.rs`):

```
let words = vec!["listen", "port", "8080"];
let mut total = 0;
thread::scope(|s| {
    let lengths = s.spawn(|| words.iter().map(|w| w.len()).sum::<usize>());
    // This handle is dropped without `join`: the scope still waits for it.
    s.spawn(|| println!("a scoped thread sees {} words", words.len()));
    total = lengths.join().unwrap();
});
println!("after the scope: {total} bytes, and `words` is ours again: {words:?}");
```

```
a scoped thread sees 3 words
after the scope: 14 bytes, and `words` is ours again: ["listen", "port", "8080"]
```

The second thread's handle was never joined, and the scope waited for it anyway. The design is the callback pattern you have seen throughout this book: the callee, not the caller, controls when the region ends. The higher-ranked `for<'scope>` (Chapter 9) adds the last piece: `'scope` is picked inside `scope`, so nothing that mentions it can leave the closure (`code/compile_fail/ch13_scope_escape.rs`):

```
pub fn escape() -> usize {
    let words = vec!["listen", "port"];
    let handle = thread::scope(|s| s.spawn(|| words.len()));
    handle.join().unwrap()
}
```

```
error: lifetime may not live long enough
  --> compile_fail/ch13_scope_escape.rs:10:36
   |
10 |     let handle = thread::scope(|s| s.spawn(|| words.len()));
   |                                -- ^^^^^^^^^^^^^^^^^^^^^^^^ returning this value
requires that `'1` must outlive `'2`
   |                                ||
   |                                |return type of closure is ScopedJoinHandle<'2, usize>
   |                                has type `&'1 Scope<'1, '_>`
   |
help: consider adding 'move' keyword before the nested closure
   |
10 |     let handle = thread::scope(|s| s.spawn(move || words.len()));
   |                                           ++++
```

The closure's return type, `ScopedJoinHandle<'2, usize>`, would have to outlive the `Scope` it came from, `'1`. The help is a false lead: with `move`, the closure owns `words`, but the handle still carries `'scope`, and the error stays (`code/compile_fail/ch13_scope_escape_move.rs`):

```
error: lifetime may not live long enough
  --> compile_fail/ch13_scope_escape_move.rs:10:36
   |
10 |     let handle = thread::scope(|s| s.spawn(move || words.len()));
   |                                -- ^^^^^^^^^^^^^^^^^^^^^^^^^^^^ returning this value
requires that `'1` must outlive `'2`
   |                                ||
   |                                |return type of closure is ScopedJoinHandle<'2, usize>
   |                                has type `&'1 Scope<'1, '_>`
```

The standard library's implementation is written with leaks in mind, too. Its counter of running threads checks for overflow, and the source comment explains when that could happen: "This can

only reasonably happen by mem::forget()'ing a lot of ScopedJoinHandles." Chapter 11 explains why an async version of the same design is much harder: there, the scope would itself be a future, and a future can be forgotten.

13.6 Checking `unsafe` code with Miri

The borrow checker cannot check `unsafe` code, and tests only check that a program produces the right output, not that it avoids undefined behavior. Miri fills part of that gap. Its README (https://github.com/rust-lang/miri) describes it as "an Undefined Behavior detection tool for Rust": it runs your program in an interpreter that tracks every allocation and every pointer, and stops with an error when an execution does something the language forbids. The kinds of errors it lists include:

- out-of-bounds memory accesses and use-after-free;
- invalid use of uninitialized data;
- not sufficiently aligned memory accesses and references;
- violation of basic type invariants (a `bool` that is not 0 or 1, for example);
- data races;
- experimentally, violations of the Stacked Borrows aliasing rules, or of the Tree Borrows rules as an alternative.

Miri is distributed for nightly toolchains only, as a `rustup` component. The README gives these commands:

```
rustup +nightly component add miri
cargo +nightly miri test
cargo +nightly miri run
```

`cargo miri` wraps Cargo, so for an example of this book you would add the usual `--example ch13_raw_iter`. Miri's own flags go in the `MIRIFLAGS` environment variable; one of them, `-Zmiri-tree-borrows`, "replaces Stacked Borrows with the Tree Borrows rules", which the README calls "even more experimental".

Two limits matter. First, Miri checks the executions it runs, and nothing else. Using the slice returned by `keep_forever` is a use-after-free, the first kind of error in the list, but only an execution that calls `keep_forever` contains it; a function that is never called, or a path that no test takes, is never checked. Second, it checks the *behavior* of your code, not its *contract*. An execution of `CovariantSlot` that only stores string literals has no undefined behavior at all, so there is nothing to find; the unsoundness shows only when a caller uses the covariance, and a test has to be written to do that. The README is explicit: "Miri does not catch every violation of the Rust specification in your program", and "Miri fundamentally cannot ensure that your code is sound." Use it as you would use a sanitizer: run your tests under it, especially tests that exercise the edges of your `unsafe` code, and keep writing the `SAFETY:` arguments that explain why the code is right.

> **NOTE**
> Miri was not installed on the machine used for this book (checked on 2026-10-12 with the nightly of 2026-10-08), so the commands above come from the README and were not run. Check the README for the current instructions.

13.7 Common confusions

- "**`unsafe` turns off the borrow checker.**" It does not. Inside an `unsafe` block every reference is still checked; `unsafe` only allows a few extra operations, such as dereferencing raw pointers. What escapes the checker is the lifetime of a reference made from a raw pointer, until a signature bounds it.
- "**`as_bytes_unbounded` is fine as long as callers are careful.**" A safe function must be safe for *every* caller the compiler accepts. If a safe caller can create a dangling reference without writing `unsafe`, the function is unsound, however unlikely the misuse.
- "**`PhantomData` is just there to silence E0392.**" It decides variance, auto traits and drop behavior. A marker that silences the error but gives the wrong variance compiles, and can be unsound (`CovariantSlot`).
- "**`Pin` makes self-referential structs safe to write.**" It keeps a value from moving; creating the self-reference is still up to you, with `unsafe`.
- "**A destructor is a guarantee.**" It is a strong habit, not a guarantee: `mem::forget`, `Box::leak` and `Rc` cycles all skip it in safe code. Soundness must never depend on it.
- "**Miri passed, so my code is sound.**" Miri found no undefined behavior in the executions it ran. Soundness is a property of every possible caller.

13.8 Summary

- Raw pointers carry no lifetime. A reference made from one (`&*ptr`, `slice::from_raw_parts`, `ptr.as_ref()`) has an *unbounded* lifetime that becomes whatever the context asks for, even `'static`.
- Bound it immediately with a signature: `fn as_bytes(&self) -> &[u8]` ties the result to `self`, and the borrow checker then enforces it on every caller (E0515 for the dangling version). `fn f<'a>(&self) -> &'a [u8]` compiles and is unsound.
- In the standard library's raw-pointer APIs, a lifetime that appears only in the result is a promise of the caller, stated in the safety documentation.
- A lifetime parameter must be used by a field (E0392). `PhantomData<X>` makes a type behave as if it held an `X`: `PhantomData<&'a T>` for a borrowing iterator, `PhantomData<&'a mut T>` or `PhantomData<Cell<T>>` for invariance, `PhantomData<fn(T)>` for contravariance. `NonNull<T>` is covariant, so a type that writes through it needs an invariant marker.

- Safe Rust cannot express a struct that borrows from itself: the constructor is E0515/E0505, and a value built in place is frozen (E0505 on a move). Store indices or ranges and borrow on demand. `Pin` protects the self-referential futures the compiler builds; it does not create self-references.
- A generic type with a `Drop` impl makes the drop check assume that `drop` uses its borrows ("borrow might be used here, when `log` is dropped"). `Vec` avoids this with `unsafe impl<#[may_dangle] T> Drop`, which is unstable (E0658); on stable, move the `Drop` into a field that does not mention `T`.
- Leaking is safe (`mem::forget`, `Box::leak`, `Rc` cycles). The pre-1.0 `thread::scoped` relied on a guard's destructor and was unsound; `thread::scope` (1.63) joins inside its own body and uses `for<'scope>` so that nothing escapes.
- Miri (nightly) detects undefined behavior in the executions it runs, including use-after-free and aliasing violations; it cannot prove soundness.

13.9 Exercises

1. **Bound it with a helper.** For `ch13_raw_buf.rs`, write an `unsafe fn bytes_at` that takes `owner: &'a RawBuf`, a start and a length, and returns a `&'a [u8]`. Give it a `# Safety` section in its documentation comment that says what the caller must promise, and a `SAFETY:` comment at the call to `from_raw_parts`. Which of the preconditions of `from_raw_parts` does the signature take care of, and which remain the caller's? *Hint:* the lifetime and the "not mutated for `'a`" requirement come from `owner`; the range must still be checked or promised.
2. **A mutable raw iterator.** Write `RawIterMut<'a, T>`, which yields `&'a mut T`. Which `PhantomData` marker gives it the variance of `slice::IterMut`, and why would `PhantomData<&'a T>` be wrong? *Hint:* `&'a mut T` is invariant in `T` (Chapter 4). Try to write a function that turns a `RawIterMut<'a, &'static str>` into a `RawIterMut<'a, &'short str>` with each marker.
3. **Variance from two fields.** In `ch13_out_slot_variance.rs`, replace the marker with `PhantomData<(&'s (), fn(&'v str))>`. Is `dangling` still rejected? Work out the variance of `OutSlot` in `'v` before you compile, then check your answer. *Hint:* `NonNull<&'v str>` is covariant in `'v` and `fn(&'v str)` is contravariant; Chapter 4 explains how the variances of several fields combine.
4. **A document without a `Range`.** Rewrite `Document` from `ch13_document_ranges.rs` so that it owns its text and offers `fn lines(&self) -> impl Iterator<Item = &str>` and `fn title(&self) -> &str`, without storing any position at all. When is recomputing better than storing an index? *Hint:* `str::lines` and `Iterator::next` on a borrow of `self.text`.
5. **A leak-proof guard?** Someone proposes a scoped-thread API that returns a guard, but makes the guard `!Send` and the constructor `unsafe`. Does this fix the problem described in *Why `thread::scope` takes a closure*? What must the caller of the `unsafe` constructor promise? *Hint:* an `Rc` cycle does not need to send anything anywhere. The promise is about every path by which the guard could be leaked.